Embedded Assignments Run Guide

This guide explains how to run each assignment in Keil step by step, what to open, what to watch, where to place breakpoints, and what result you should expect.

> **Important note:** Assignment 03, Assignment 05, and Assignment 07 were already runtime-checked in Keil and their steps below are verified. Assignment 06, Assignment 04, and Assignment 02 are based on the current code in the workspace and are prepared as the next debug flow to follow.

Recommended Order

1. Assignment 03
2. Assignment 05
3. Assignment 07
4. Assignment 06
5. Assignment 04
6. Assignment 02

Standard Debug Windows

For all assignments, keep these windows open unless noted otherwise:

- source window showing `main.c`
- `Watch 1`
- `Call Stack + Locals`
- `Registers`

`Registers` is useful, but the most important windows are `main.c` and `Watch 1`.

Assignment 03

Project file: `Hassabou-Assignments/Assignment-03/Assignment-03.uvprojx`

Build

1. Open the project in Keil.
2. Click `Rebuild`.
3. Expected result: `0 Error(s), 0 Warning(s)`.

Watch Variables

- `g_interruptCount`
- `g_taskWakeCount`
- `g_lastNotificationTick`
- `g_actionTaskHandle`

Breakpoints

- the line `g_interruptCount++;` inside `GPIOF_Handler()`
- the line `g_taskWakeCount += pendingNotifications;` inside `vActionTask()`

Run Steps

1. Start a debug session.
2. Press `Run`.
3. Use the LaunchPad / Port F simulator window.
4. Press `SW1` once.
5. The first stop should be in `GPIOF_Handler()` at `g_interruptCount++;`.
6. Press `F10` once.
7. Expected: `g_interruptCount` becomes `1`.
8. Press `F5`.
9. The next stop should be in `vActionTask()` at `g_taskWakeCount += pendingNotifications;`.
10. Press `F10` once.
11. Expected: `g_taskWakeCount` becomes `1` and `g_lastNotificationTick` changes from `0`.

Assignment 05

Project file: `Hassabou-Assignments/Assignment-05/Assignment-05.uvprojx`

Build

1. Open the project in Keil.
2. Click `Rebuild`.
3. Expected result: `0 Error(s), 0 Warning(s)`.

Watch Variables

- `gInterruptsFired`
- `gSemaphoreAccepted`
- `gDroppedInterrupts`
- `gTasksProcessed`
- `gTriggerBursts`
- `gHandlerBusy`

Breakpoints

- the line `gInterruptsFired++;` inside `GPIOF_Handler()`
- the line `gTasksProcessed++;` inside `vHandlerTask()`

Run Steps

1. Start a debug session.
2. Press `Run`.
3. Do not click anything in the simulator. This project triggers interrupts automatically in software.
4. The first stop should be in `GPIOF_Handler()` at `gInterruptsFired++;`.
5. Press `F10` once.
6. Expected: `gInterruptsFired` becomes `1`.
7. Press `F5`.
8. The next stop should be in `vHandlerTask()` at `gTasksProcessed++;`.
9. Press `F10` once.
10. Expected: `gTasksProcessed` becomes `1` and `gHandlerBusy` stays `1`.
11. Press `F5` again.
12. The next stop should return to the ISR while the handler is still busy.
13. Step through the ISR until after the `xSemaphoreGiveFromISR()` decision.
14. Keep watching `gSemaphoreAccepted` and `gDroppedInterrupts`.
15. Final expected proof: `gDroppedInterrupts` becomes `1` or greater.

Assignment 07

Project file: `Hassabou-Assignments/Assignment-07/Assignment-07.uvprojx`

Watch Variables

- `gInterruptsFired`
- `gTokensQueued`
- `gTokensProcessed`
- `gBurstCounter`
- `gHandlerActive`

For the required Logic Analyzer evidence, also add the signal `gTokensProcessed` to the Logic Analyzer window.

Breakpoints

- the line `gInterruptsFired++;` inside `GPIOF_Handler()`
- the line `gTokensProcessed++;` inside `vHandlerTask()`

Run Summary

1. Build and start debug.
2. Run without clicking anything manually.
3. First stop: ISR. Step once and expect `gInterruptsFired = 1` and `gTokensQueued = 1`.
4. Continue to the handler stop. Step once and expect `gTokensProcessed = 1`.
5. Continue until all three interrupts occur.
6. Final expected behavior: three interrupts, three queued tokens, and a staircase increase in `gTokensProcessed`.
7. Open `View` -> `Analysis Windows` -> `Logic Analyzer`.
8. Click `Setup...` and add `gTokensProcessed`.
9. Click `Clear`, then run again and let the burst complete.
10. Expected Logic Analyzer result: `gTokensProcessed` rises from `0` to `1`, then `2`, then `3` in separate steps.
11. If a second burst happens, the same step pattern repeats and the count continues increasing.

Success means three distinct interrupts occur, no tokens are lost, and the Logic Analyzer shows the task processing those tokens one by one instead of one single jump.

Assignment 06

Project file: `Hassabou-Assignments/Assignment-06/Assignment-06.uvprojx`

Watch Variables

- `gActiveResources`
- `gMaxObservedResources`
- `gAcquireCount[0..4]`
- `gReleaseCount[0..4]`
- `gTaskUsingResource[0..4]`

Breakpoints

- the line `gActiveResources++;`
- the line `gActiveResources--;`

Run Summary

1. Build and start debug.
2. Run and let worker tasks compete automatically.
3. Stop when a resource is acquired, then step once.
4. Expected: `gActiveResources` increases.
5. Continue until two tasks are active.
6. Expected final proof: `gMaxObservedResources` never exceeds `2`.

Assignment 04

Project file: `Hassabou-Assignments/Assignment-04/Assignment-03.uvprojx`

Watch Variables

- `gInterruptCount`
- `gPendSvRequests`
- `gHighTaskRuns`
- `gLowTaskIterations`
- `gLastIsrTick`

- `gLastHighTaskTick`
- `gObservedLatencyTicks`

Breakpoints

- the line `gInterruptCount++;` inside `GPIOF_Handler()`
- the line `gHighTaskRuns++;` inside `vHighPriorityTask()`

Run Summary

1. Build and start debug.
2. Run without clicking anything manually.
3. Record behavior with `USE_ISR_YIELD = 1`.
4. Then change it to `0`, rebuild, and repeat.
5. Compare `gObservedLatencyTicks` between both runs.

Assignment 02

Project file: `Hassabou-Assignments/Assignment-02/Assignment-02.uvprojx`

Additional Windows

- `Memory 1`
- Disassembly only if needed

Watch Variables

- `g_ui32SysTickCount`
- `g_restoreAttemptCount`
- `g_currentTaskId`
- `g_nextTaskId`
- `g_contextCorruptionDetected`
- `g_expectedPc`
- `g_observedPc`
- `g_faultSignature`
- `g_currentSp`
- `g_restoredSp`
- `g_faultyRestoreSp`

- `g_lastDecodedFrame.pc`
- `g_lastDecodedFrame.xpsr`

Breakpoints

- the line `g_restoreAttemptCount++;` inside `AttemptContextRestore()`
- the corruption-check `if` that compares the decoded frame values

Run Summary

1. Build and start debug.
2. Run until the restore attempt occurs.
3. Step through the frame decode.
4. Compare `g_expectedPc`, `g_observedPc`, `g_restoredSp`, and `g_faultyRestoreSp`.
5. Expected final proof: `g_contextCorruptionDetected = 1` and the frame values no longer match a valid restore.